INGENIERÍA DE SEGURIDAD US
E.T.S. Ingeniería Informática. Universidad de Sevilla
Avda. Reina Mercedes s/n. 41012 Sevilla
Tlf/Fax 954 557 139 **E-mail** lsi@lsi.us.es **Web** www.lsi.us.es
INSEGUS

PAI 4. DevSecOps - ASEGURAMIENTO DE LA CADENA DE SUMINISTRO EN EL DESARROLLO DE APLICACIONES WEB

Introducción

Una empresa de desarrollo de aplicaciones web, cliente de INSEGUS, ha sufrido recientemente varios incidentes de seguridad. El análisis posterior apunta a vulnerabilidades que llegaron a producción sin ser detectadas: fallos en el propio código, dependencias de terceros con vulnerabilidades conocidas y configuraciones inseguras de la infraestructura. Para mejorar su **postura de seguridad**, la empresa ha decidido adoptar un enfoque **DevSecOps** apoyado en herramientas de automatización. Actualmente desarrolla siguiendo un pipeline de integración y despliegue continuos (CI/CD) como el de la Figura 1.

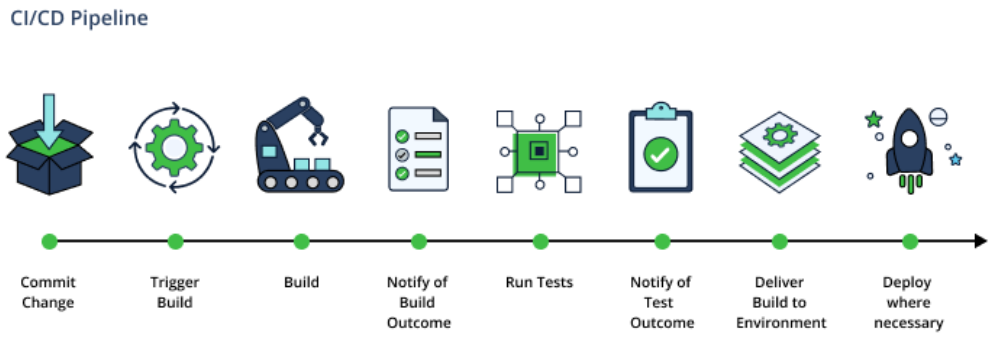


Figura 1: Pipeline CI/CD actual de la empresa

El responsable de la empresa se pregunta si los incidentes se deben a esta forma de trabajar y, en particular, a cómo se comprueba el cumplimiento de las políticas de seguridad de la empresa: en el pipeline actual, ninguna etapa verifica la seguridad del código, de sus dependencias ni de su configuración.

El escenario se complica porque la empresa está migrando de aplicaciones monolíticas a aplicaciones basadas en **microservicios** (Figura 2), lo que aumenta la superficie de ataque.

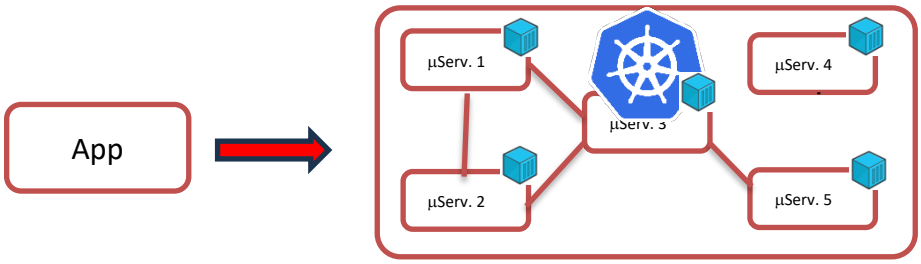


Figura 2: Aplicación basada en microservicios desplegada en Kubernetes

Además, estos microservicios se ejecutan en **contenedores**, se despliegan en nubes públicas y se gestionan con **Kubernetes**. Cada una de estas capas añade componentes, configuraciones y dependencias que pueden contener vulnerabilidades y, por tanto, nuevos riesgos que hay que vigilar.

En cada desarrollo participan muchas personas y el pipeline ya está automatizado, así que revisar la seguridad a mano no es viable. Por ello, la empresa quiere automatizar también el análisis y la mitigación de estos riesgos, integrando pruebas de seguridad a lo largo de todo el pipeline, desde que el desarrollador sube el código hasta el despliegue en producción. Es lo que propone el esquema **DevSecOps** de la Figura 3.

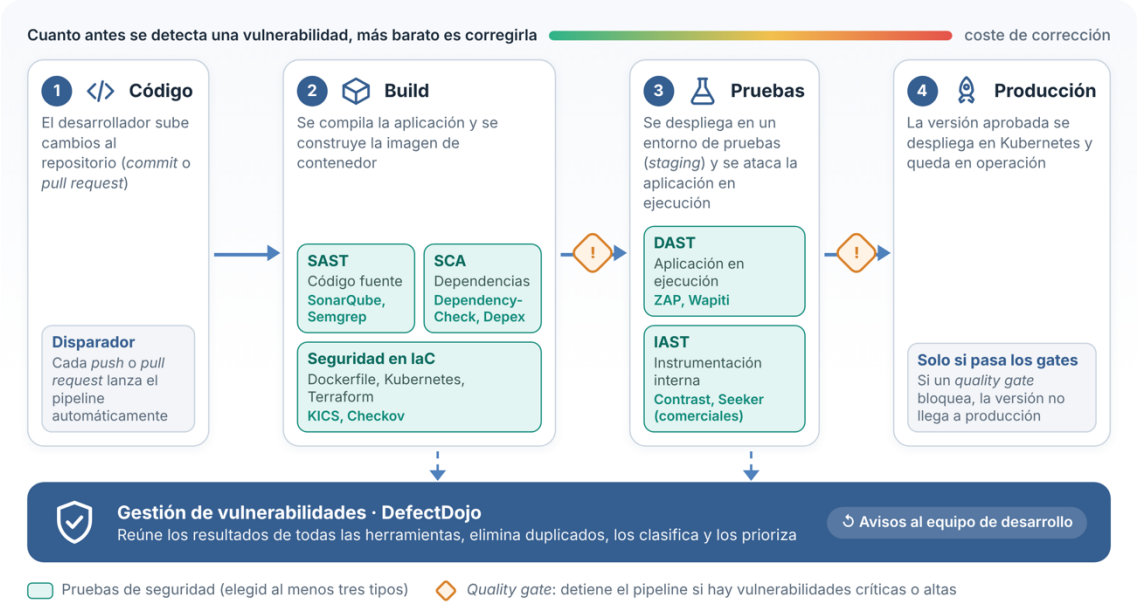


Figura 3: Pipeline DevSecOps propuesto

La empresa contempla los tipos de herramientas de la Tabla 1. Sus resultados se pueden centralizar en una única plataforma de gestión de vulnerabilidades, como *DefectDojo*. Así la seguridad se gestiona como código (*Security as Code*), los fallos se detectan en etapas tempranas, cuando corregirlos es mucho más barato que en producción, y se acortan los ciclos de entrega de código seguro.

Tipo	Qué analiza	Etapa del pipeline
SCA (*Software Composition Analysis*)	Dependencias y librerías de terceros con vulnerabilidades conocidas (CVE) o licencias problemáticas	Commit y build
SAST (*Static Application Security Testing*)	El código fuente, sin ejecutarlo, en busca de patrones inseguros (inyecciones, credenciales en el código…)	Commit y build
DAST (*Dynamic Application Security Testing*)	La aplicación en ejecución, atacándola desde fuera como lo haría un atacante	Tras desplegar en un entorno de pruebas
IAST (*Interactive Application Security Testing*)	La aplicación en ejecución, instrumentada desde dentro mientras se ejecutan pruebas	Pruebas en un entorno de pruebas
Seguridad en IaC (*Infrastructure as Code*)	Ficheros de infraestructura (Dockerfile, manifiestos de Kubernetes, Terraform…) en busca de configuraciones inseguras	Commit y build

Tabla 1: Tipos de herramientas de pruebas de seguridad

Objetivos del proyecto

Se propone a cada *Security Team* **poner en marcha un pipeline DevSecOps, lo más automatizado posible, que entregue un producto software funcional y seguro**. Los objetivos son los siguientes:

1. **Diseñar el pipeline CI/CD y, después, montarlo** para una aplicación de prueba alojada en un repositorio de código (GitHub, GitLab u otro SCM). La aplicación debe ser una aplicación web intencionadamente vulnerable (ver «Aplicaciones de prueba recomendadas»), para que las herramientas tengan vulnerabilidades que detectar. **Antes de implementarlo**, el *Security Team* debe dibujar su diseño en dos diagramas, que le ayudarán a reflexionar sobre qué se comprueba, dónde y qué ocurre cuando algo falla:
   a. **Diagrama del pipeline DevSecOps:** las etapas en orden; la herramienta de seguridad que se ejecuta en cada etapa; el informe que genera y cómo llega a la herramienta de gestión de vulnerabilidades; y la posición de cada *quality gate*, indicando si es bloqueante o está en modo aviso y con qué umbral.
   b. **Diagrama de flujo ante una vulnerabilidad:** qué ocurre desde que una herramienta detecta una vulnerabilidad hasta que se resuelve. Debe mostrar las decisiones (¿supera el umbral?, ¿la etapa es bloqueante?, ¿es un falso positivo?), qué hace el pipeline en cada caso (continúa o se detiene), a quién se avisa y cómo se registra y se trata el hallazgo en la herramienta de gestión de vulnerabilidades.

   Una vez montado el pipeline, los diagramas deben corresponderse con lo implementado: cada etapa y cada herramienta del diagrama debe estar en los ficheros del pipeline, y viceversa. Si durante la implementación cambia el diseño, se actualizan los diagramas y se explica en el informe por qué cambió. Las Figuras 4 y 5 muestran un ejemplo de cada diagrama para un caso distinto (WebGoat en GitLab CI/CD): sirven de referencia del nivel de detalle esperado, no como plantilla para copiar. Los diagramas de cada equipo deben reflejar su propio pipeline.

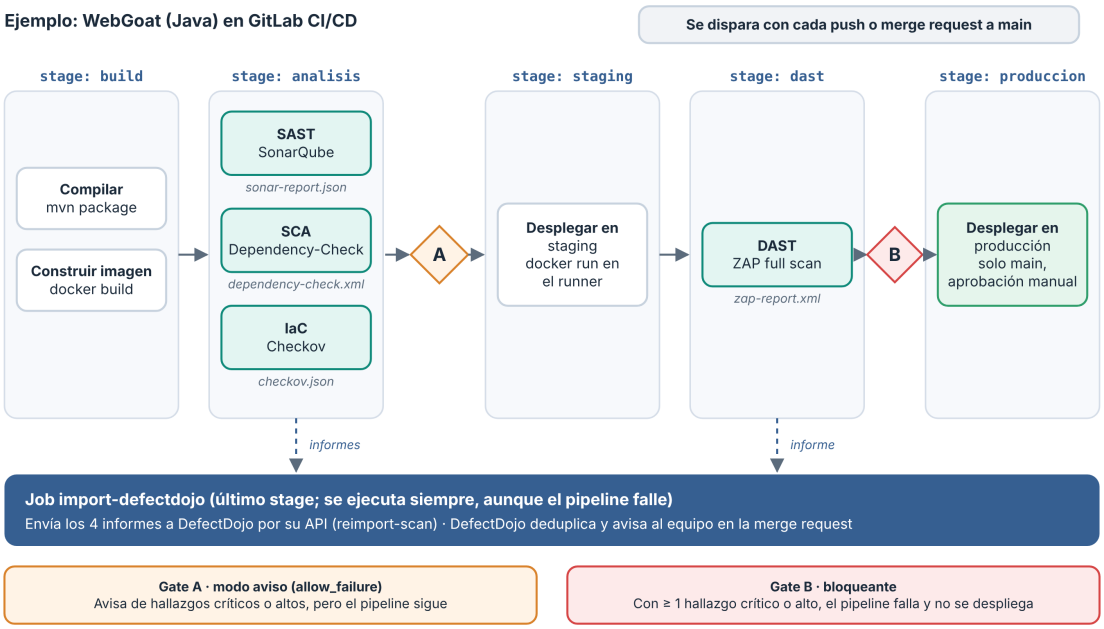


Figura 4: Ejemplo de diagrama del pipeline (objetivo 1.a)

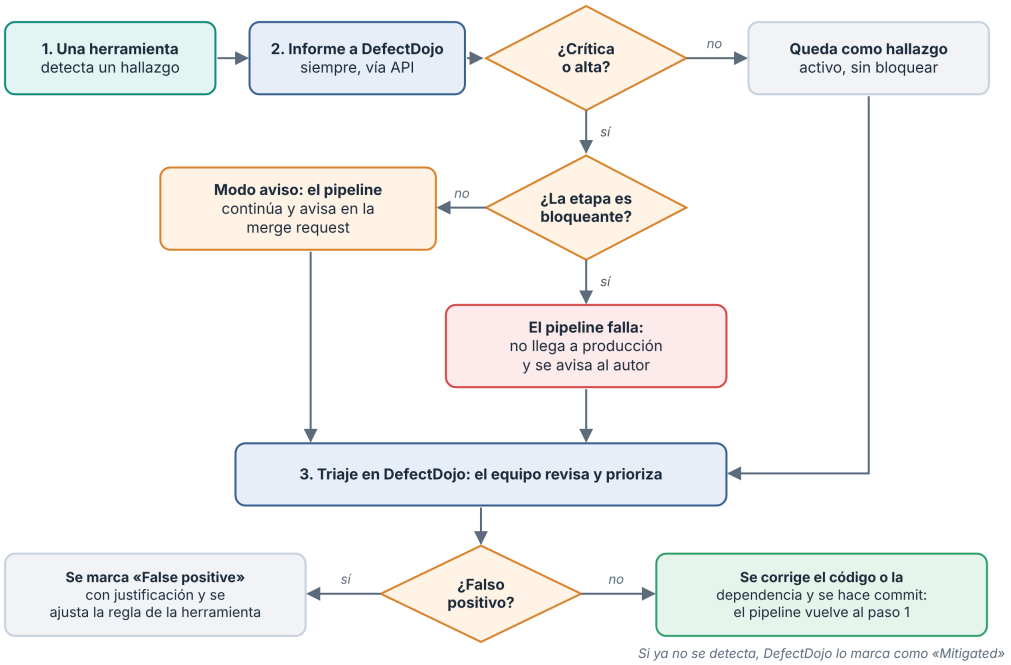


Figura 5: Ejemplo de diagrama de flujo ante una vulnerabilidad (objetivo 1.b)

2. **Seleccionar al menos tres herramientas de pruebas de seguridad de tipos distintos** (SCA, SAST, DAST, IAST o seguridad en IaC) y justificar la elección según la tecnología de la aplicación.
3. **Integrar las herramientas en el pipeline**, de forma que se ejecuten automáticamente con cada cambio del código (por ejemplo, en cada *push* o *pull request*) y cada una en la etapa que le corresponda.
4. **Configurar criterios de seguridad que bloqueen el pipeline** (*quality gates*); por ejemplo, que el pipeline falle si se detecta una vulnerabilidad crítica o alta. Hay que demostrar que cada herramienta detecta al menos una vulnerabilidad de la aplicación y que el pipeline reacciona como se ha configurado. Como la aplicación es vulnerable, un *quality gate* que bloquee en la etapa de build impediría llegar a las pruebas dinámicas. Por eso, algunas etapas pueden configurarse en **modo aviso**, que informa de los hallazgos sin

detener el pipeline (por ejemplo, con *continue-on-error* en GitHub Actions o *allow_failure* en GitLab CI/CD). **Al menos un quality gate debe bloquear el pipeline de verdad.**

5. **Centralizar los resultados en una herramienta de gestión de vulnerabilidades** que permita integrar, deduplicar, clasificar y priorizar los hallazgos de todas las herramientas. Se recomienda *DefectDojo*, aunque se puede usar cualquier otra similar, justificando la elección. Los resultados deben enviarse **automáticamente desde el pipeline** (por ejemplo, mediante la API de la herramienta), no subirse a mano.

Herramientas recomendadas

Las listas siguientes son orientativas: se puede usar cualquier otra herramienta, justificando su elección. Se recomiendan herramientas libres o con versión gratuita.

Aplicaciones de prueba recomendadas

Aplicaciones web intencionadamente vulnerables, pensadas para practicar. Se puede usar otra si contiene vulnerabilidades detectables por las herramientas elegidas.

- **OWASP Juice Shop** (Node.js y Angular; incluye Dockerfile): https://owasp.org/www-project-juice-shop/
- **OWASP WebGoat** (Java): https://owasp.org/www-project-webgoat/
- **DVWA** (PHP): https://github.com/digininja/DVWA
- **OWASP NodeGoat** (Node.js): https://github.com/OWASP/NodeGoat

Pipelines CI/CD

- **GitHub Actions**: https://docs.github.com/actions
- **GitLab CI/CD**: https://docs.gitlab.com/ci/
- **Jenkins**: https://www.jenkins.io/doc/

Software Composition Analysis (SCA)

- **Depex**: https://github.com/GermanMT/depex
- **GitHub Dependabot**: https://docs.github.com/code-security/dependabot
- **OWASP Dependency-Check**: https://owasp.org/www-project-dependency-check/
- **Snyk** (versión gratuita con límites): https://snyk.io/
- **Sonatype Nexus Lifecycle** (comercial): https://www.sonatype.com/products/open-source-security-dependency-management

Static Application Security Testing (SAST)

- **Catálogo de herramientas de OWASP**: https://owasp.org/www-community/Source_Code_Analysis_Tools
- **SonarQube Community Build**: https://www.sonarsource.com/open-source-editions/sonarqube-community-edition/
- **Semgrep Community Edition**: https://github.com/semgrep/semgrep
- **CodeQL** (gratuito en repositorios públicos de GitHub): https://codeql.github.com/

Dynamic Application Security Testing (DAST)

El análisis dinámico necesita la aplicación en ejecución. Puede levantarse dentro del propio pipeline con Docker (por ejemplo, como contenedor de servicio del *job*) y analizarse en *localhost*.

- **Catálogo de herramientas de OWASP**: https://owasp.org/www-community/Free_for_Open_Source_Application_Security_Tools

- **ZAP** (dispone de imágenes Docker y acciones de GitHub para automatizarlo): https://www.zaproxy.org/
- **Wapiti**: https://wapiti-scanner.github.io/
- **Burp Suite Community**: https://portswigger.net/burp/communitydownload. Útil para pruebas manuales; su versión gratuita no permite automatizar escaneos en el pipeline.

Interactive Application Security Testing (IAST)

Las herramientas IAST disponibles son comerciales. Pueden contar como uno de los tres tipos del objetivo 2 si se consigue una licencia de prueba; si no, basta con elegir otros tipos.

- **Contrast Security**: https://www.contrastsecurity.com/
- **Seeker** (Black Duck): https://www.blackduck.com/

Seguridad en Infrastructure as Code (IaC)

Detectan errores de configuración, vulnerabilidades y malas prácticas en ficheros como Dockerfile, manifiestos de Kubernetes, Terraform, CloudFormation o Ansible.

- **KICS**: https://kics.io/
- **Trivy**: https://trivy.dev/
- **Checkov**: https://www.checkov.io/

Gestión de vulnerabilidades

DefectDojo importa los informes de la mayoría de las herramientas anteriores, también a través de su API, lo que permite enviarle los resultados desde cualquier pipeline.

- **DefectDojo**: https://defectdojo.com/community
- **Documentación de DefectDojo**: https://docs.defectdojo.com/
- **Código fuente y despliegue local con Docker Compose**: https://github.com/DefectDojo/django-DefectDojo
- **Integración con Jenkins**: https://plugins.jenkins.io/defectdojo/
- **Integración con GitLab**: https://gitlab.com/to-be-continuous/defectdojo/

Uso responsable

- Las herramientas de análisis dinámico y los escáneres de vulnerabilidades **solo pueden usarse contra la aplicación desplegada por el propio Security Team**. Analizar sistemas de terceros sin autorización puede constituir un delito.
- Las credenciales (por ejemplo, la clave de la API de DefectDojo) deben guardarse como **secretos del pipeline**, nunca en el repositorio. Si el código entregado contiene credenciales, se penalizará.

Normas del entregable

- El plazo de entrega finaliza el **miércoles 2 de diciembre de 2026 a las 23:59**.
- Cada *Security Team* debe entregar, a través de la actividad preparada en la Enseñanza Virtual, un archivo **PAI4-STX.zip** (donde X es el número del *Security Team*) que contenga:
    - **Informe técnico en PDF (máximo 15 páginas, sin contar portada ni anexos)** que describa: las herramientas seleccionadas y por qué; el pipeline CI/CD diseñado, con los dos diagramas del objetivo 1; cómo se ha integrado cada herramienta y qué *quality gates* se han configurado; los resultados obtenidos (vulnerabilidades detectadas por cada herramienta y su gestión en la herramienta de gestión de vulnerabilidades). El informe debe incluir el enlace

al repositorio; si es privado, hay que dar acceso al profesorado. Los anexos solo pueden contener evidencias.
    - **Código y configuración**: ficheros de definición del pipeline (por ejemplo, .github/workflows/*.yml, .gitlab-ci.yml o Jenkinsfile), scripts y configuraciones de las herramientas.
    - **Evidencias**: logs de las ejecuciones del pipeline y capturas de los resultados, incluida al menos una ejecución que falle por un *quality gate* y los hallazgos registrados en la herramienta de gestión de vulnerabilidades.
- Las entregas fuera de plazo se penalizarán con un **10 % de la nota del proyecto por cada día de retraso**, hasta agotar la puntuación.
- La entrega solo es válida a través de la actividad de la Enseñanza Virtual.